Digital Signatures



CS 161 Spring 2026 - Lecture 14

Announcements

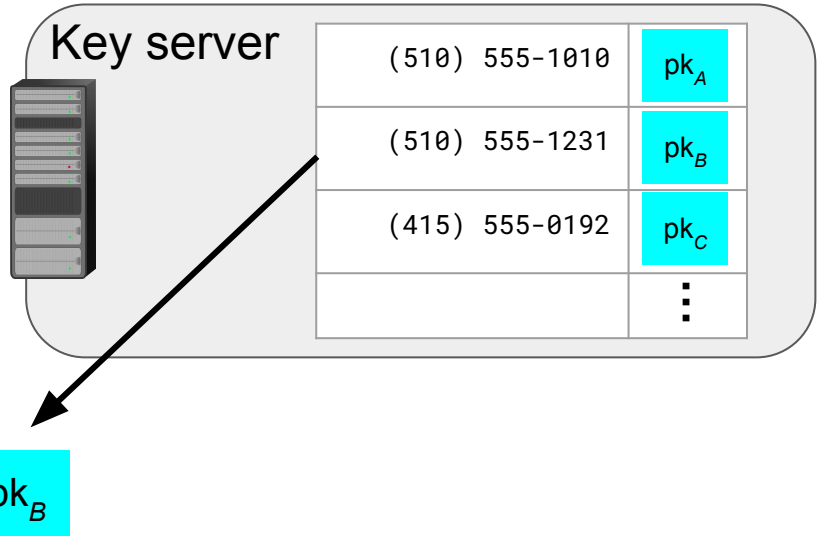
- Thank you for the feedback!
  - henrycg.com/feedback
- Notes to self:
  - Make sure Zoom screen capture works
  - Repeat the questions

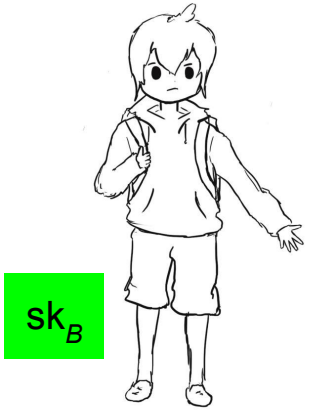
Cryptography roadmap

	Secret-key crypto	**Public-key crypto**
Encryption	✅ One-time pad ✅ CPA security ✅ Counter-mode encryption ✅ CBC encryption ✅ Authenticated encryption	✅ Key exchange (e.g., Diffie-Hellman) ✅ Public-key encryption (e.g., ElGamal)
Authentication	✅ Message auth. codes ("MACs")	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) ✅ Hash functions (e.g., SHA2)	✅ Discrete-log problem

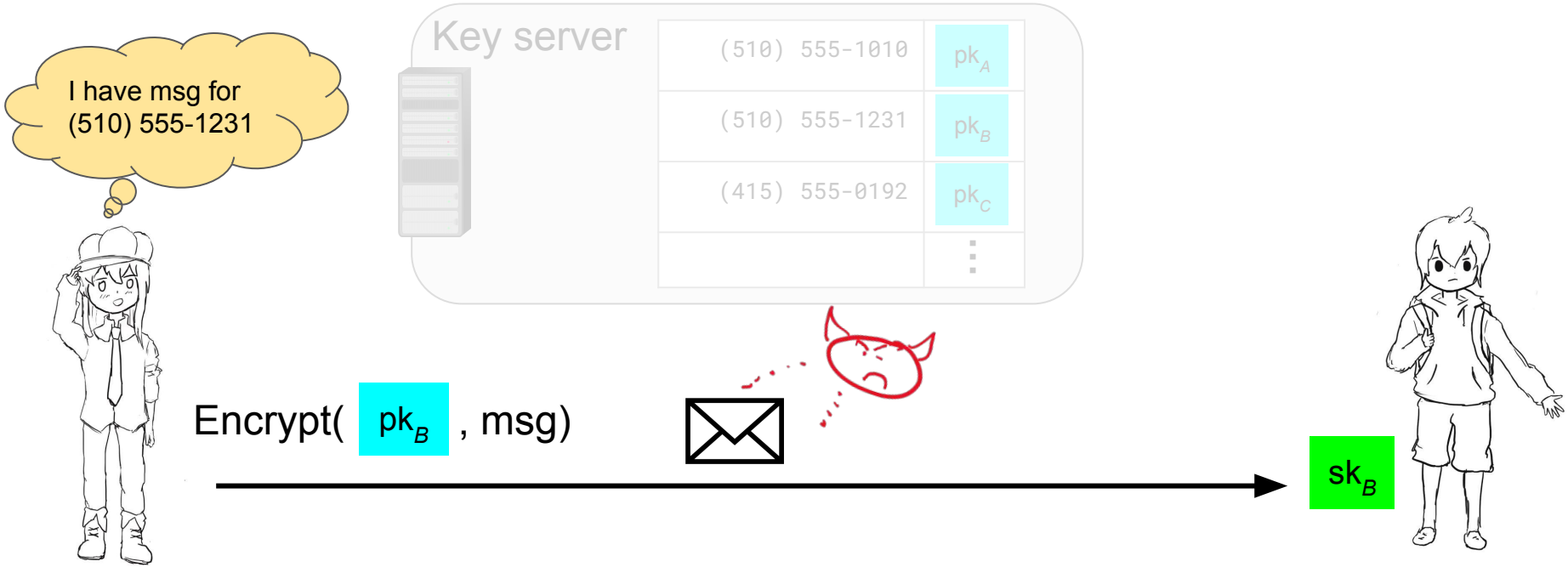
Last time: Public-key encryption



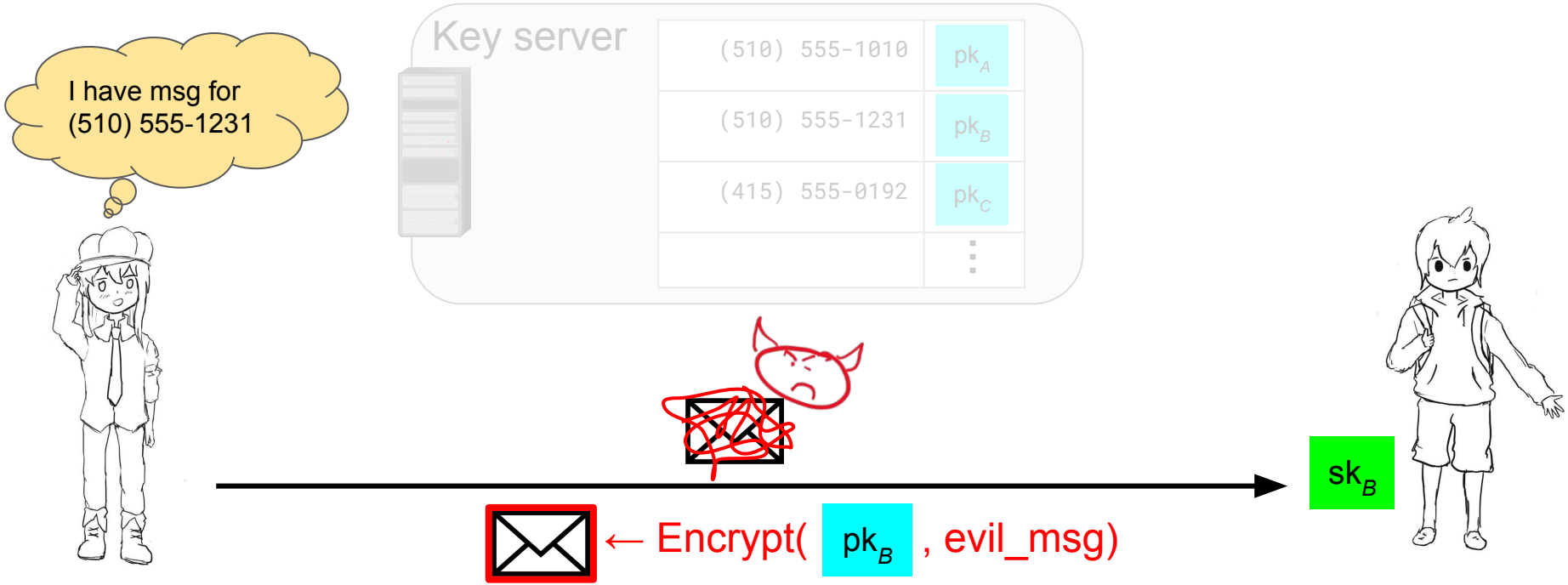




Last time: Public-key encryption



How to protect message integrity?



Integrity protection

If Alice and Bob have a shared secret:

- They can use a message authentication code (MAC)

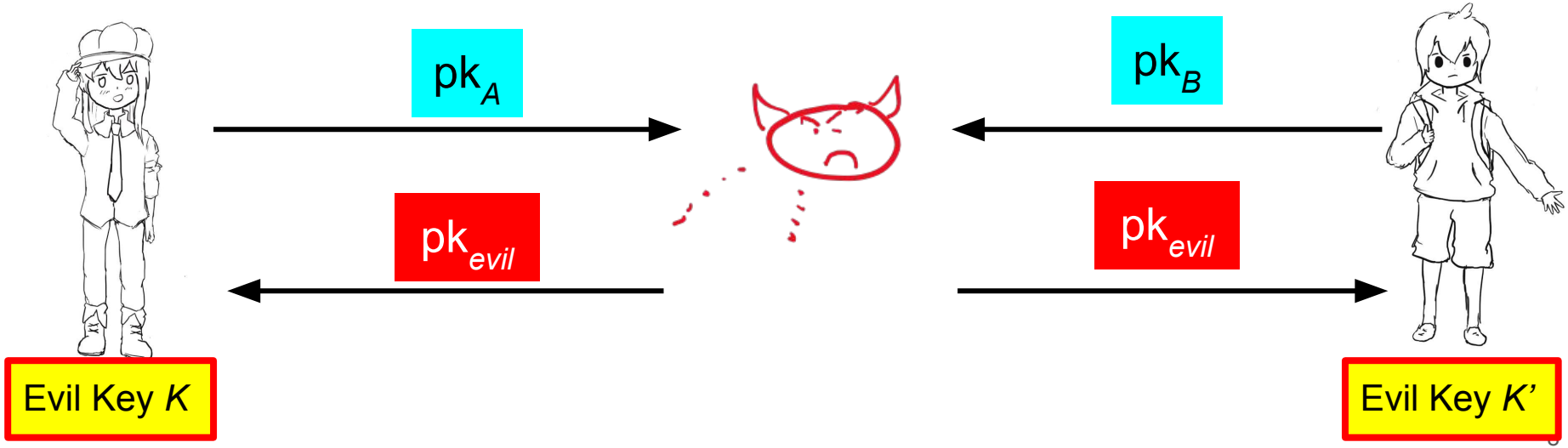
But Alice and Bob do not have a shared secret!

Bad idea:

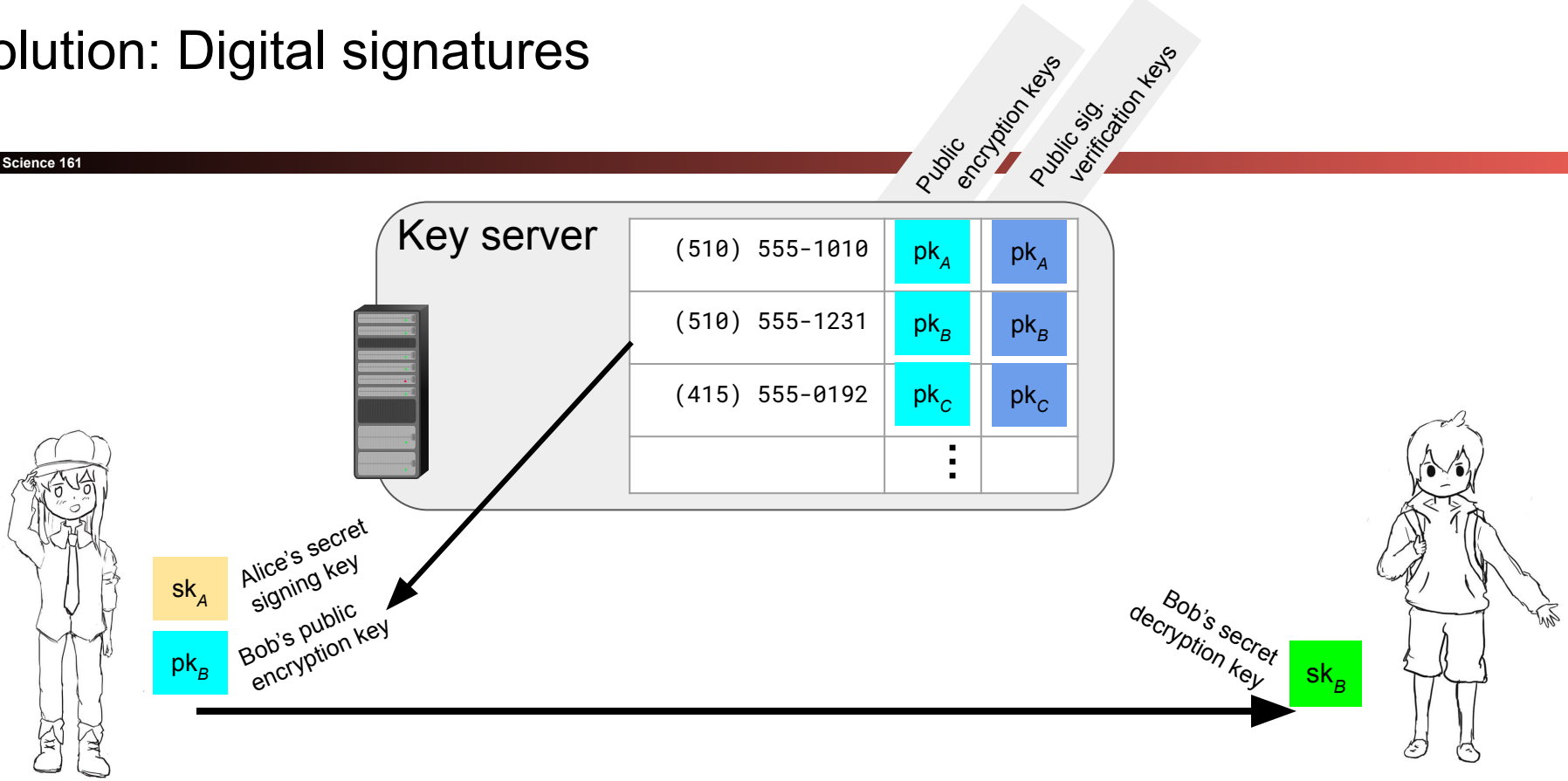
- Use DH key exchange to agree on a shared secret
- Then use a MAC?

⇒ DH key exchange does not protect integrity!

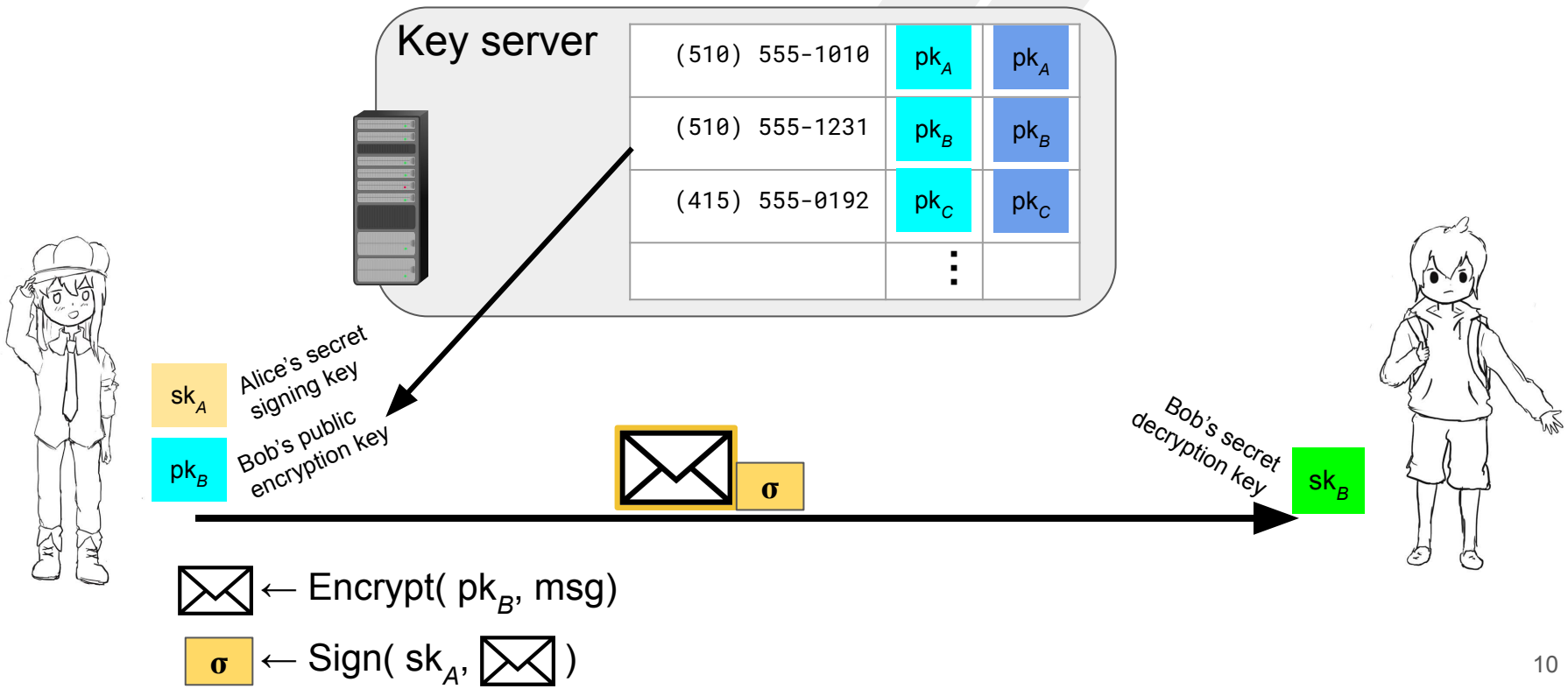
Unauthenticated key exchange does not protect against active attacks!



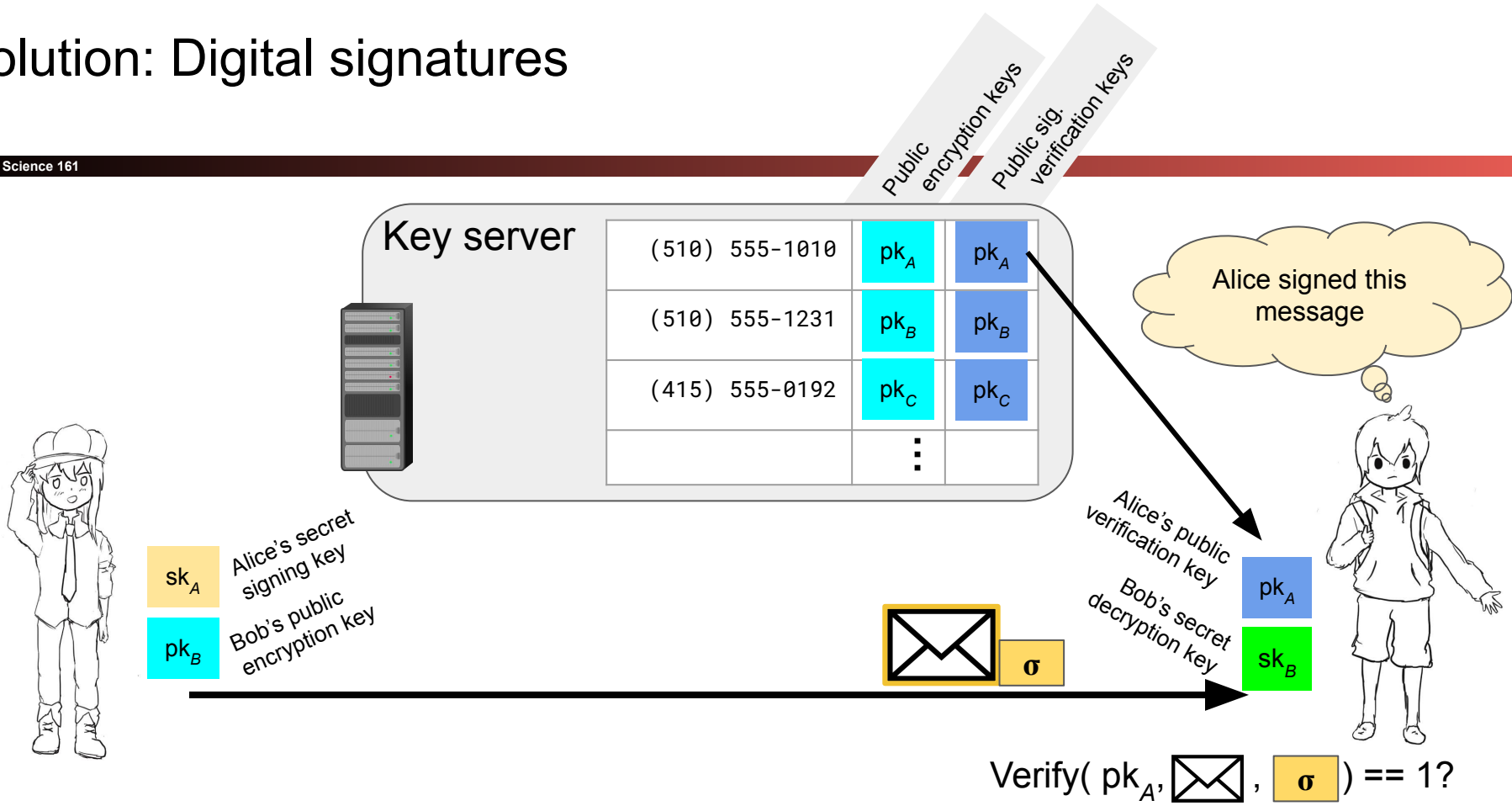
Solution: Digital signatures



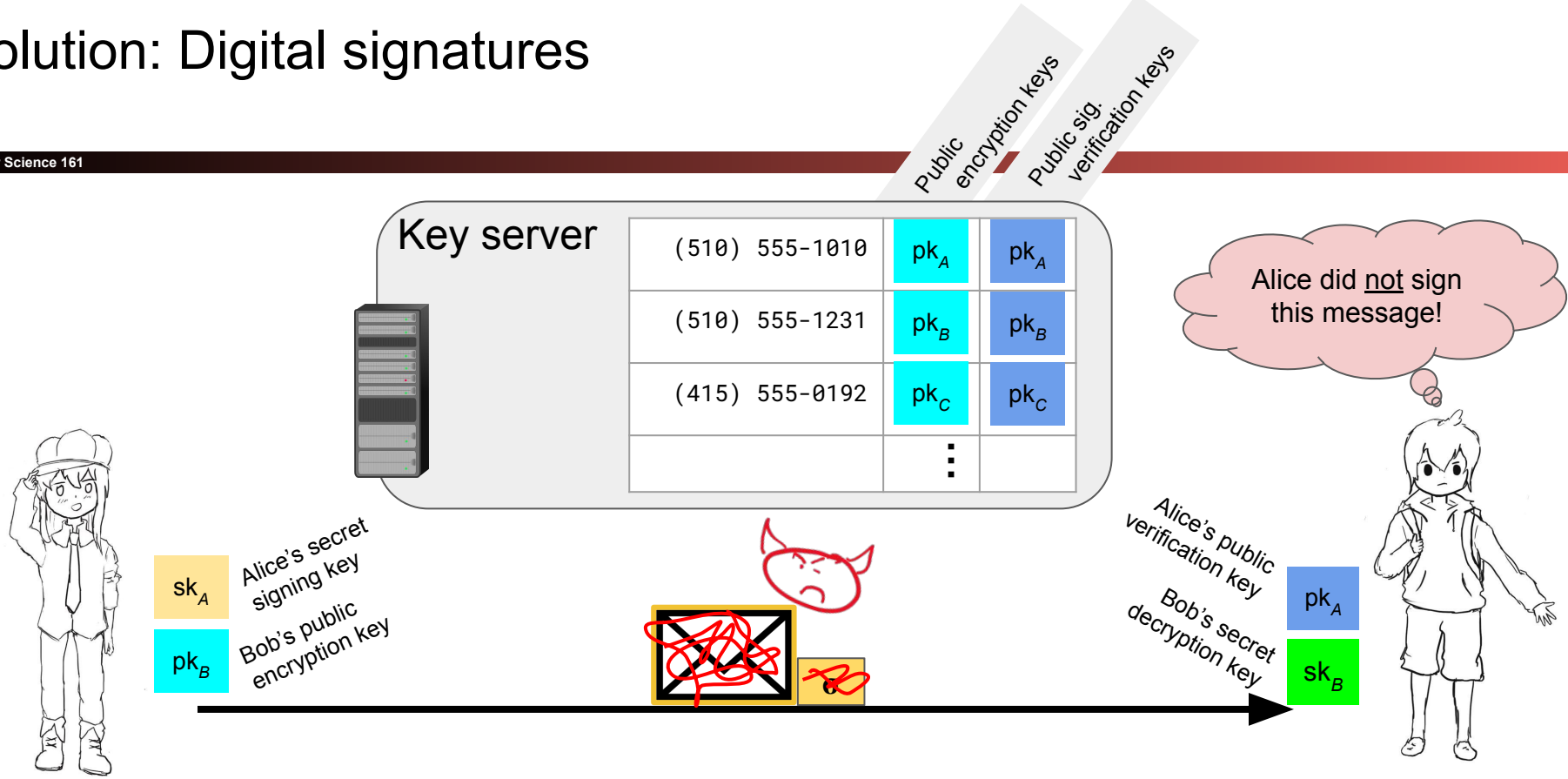
Solution: Digital signatures



Solution: Digital signatures



Solution: Digital signatures



Digital signatures: The public-key analogue of a MAC

	To encrypt, need…	To decrypt, need…
Secret-key encryption	Secret key	Secret key
Public-key encryption	Public key	Secret key

	To sign, need…	To verify, need…
Message auth. code (MAC)	Secret key	Secret key
Digital signature	Secret key	Public key

Digital signatures: Syntax

Three efficient algorithms:

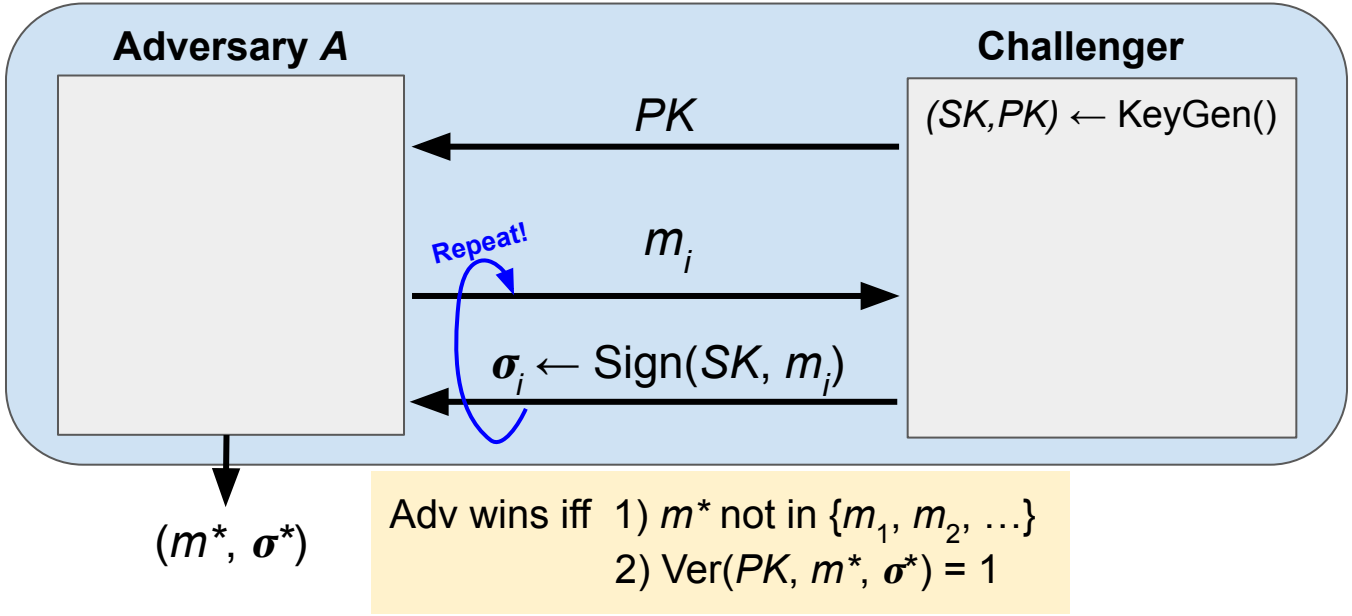
- KeyGen() → (*SK*, *PK*) // Generate keypair. Also *use VK* for public verif. key
- Sign(*SK*, *M*) → **σ** // Sign with secret key
- Verify(*PK*, *M*, **σ**) → {0,1} // Verify with public key

Properties:

- **Correctness**: If signer signs, verifier accepts
  - For all (*SK, PK*) ← KeyGen() and messages *M*, we have: Verify(*PK*, *M*, Sign(*SK*, *M*)) = 1
- **Security**:
  – Attacker gets public key and signatures on many messages of its choice
  – Still, hard for attacker to produce a valid signature on a *new* message

Digital-Signature Security Game

EUF-CMA ("Existential unforgeability under chosen-message attack")



A digital-signature scheme is *secure* if,
for all "efficient" adversaries *A*, Pr[*A* wins in Sig. game] ≤ "negligible."

Analog signatures vs. digital signatures

- **Analog:** Signature appended to message
  - Easy to cut and paste a signature from one doc to another
- **Digital:** Signature is a *function* of the message
  - Cannot cut and paste signature from one message to another

- **Analog:** If you see a signature on one message, you can produce many signatures on many messages
- **Digital:** Seeing many signatures does not help you forge new ones

Bad news: There is no “best” signature scheme

	Assumption type	Ver. key size	Sig. size	Sign()/sec	Verify()/sec	PQ secure?
ECDSA-256 (also: EdDSA)	Discrete log	32 b	64 b	77,000	25,000	N
RSA-2048	Factoring	256 b	256 b	2,300	90,000	N
SPHINCS+ 128	One-way fn (Hash)	32 b	800 b	5	750	Y
MLDSA-44	Lattice	1,312 b	2,420 b	9,000	25,000	Y
And there are many more…						

Signatures in practice

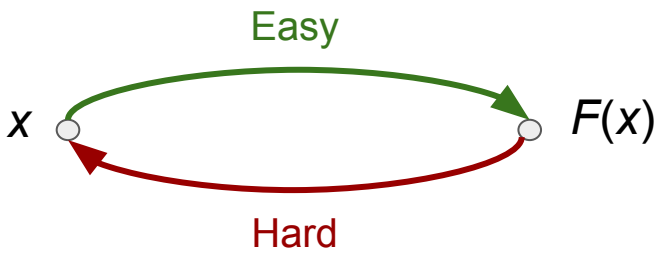
- RSA signatures are widespread
  - First to market
  - Baked into websites' "public-key certificates" and users' SSH keys = Hard to change
- ECDSA signatures are increasingly popular
  - Better than RSA in every way, except verification time
  - Default for OpenSSH `ssh-keygen` since Oct. 2023
- Post-quantum security
  - There are quantum poly-time attacks on both RSA and ECDSA
  - NIST has announced three new "post-quantum" signature schemes
    - ML-DSA, FN-DSA, SLH-DSA

Constructing Digital Signatures



Building block: Trapdoor one-way permutation

One-way function



Trapdoor one-way permutation

Building block: Trapdoor one-way permutation

A trapdoor one-way permutation with input space $\boldsymbol{X}$ is three efficient algorithms:

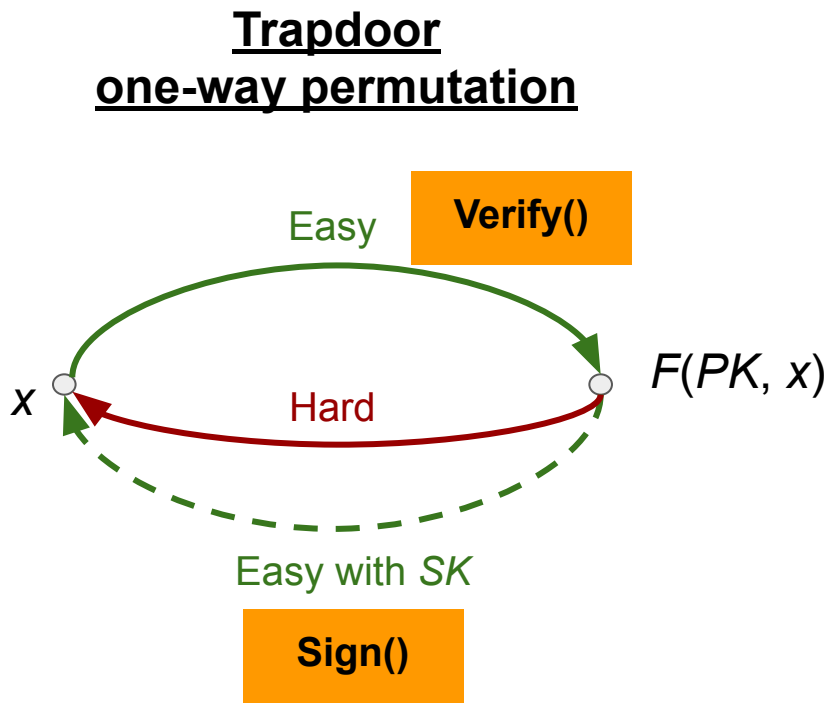
- KeyGen() $\rightarrow$ (*SK, PK*)
- F(*PK*, $x \in \boldsymbol{X}$) $\rightarrow x' \in \boldsymbol{X}$
- Invert(*SK*, $x' \in \boldsymbol{X}$) $\rightarrow x \in \boldsymbol{X}$

Properties:

- **Correctness:** Invert(*SK*, •) computes the inverse of F(*PK*, •)
  For all (*SK, PK*) $\leftarrow$ KeyGen(), for all $x \in \boldsymbol{X}$, Invert(*SK*, F(*PK*, x)) = x.

- **Security:** F(*PK*, •) is hard to invert on *random* inputs.
  For (*SK, PK*) $\leftarrow$ KeyGen(), Pr[A(*PK*, F(*PK*, *x*) = x : x $\leftarrow_R \boldsymbol{X}$] $\leq$ "negligible."

Idea: Signature from trapdoor one-way permutation



Signatures from trapdoor one-way permutation

Parameters: – Trapdoor OWP over space ***X*** (OWP.KeyGen, OWP.F, OWP.Invert)
– Cryptographic hash function Hash : {0,1}* → ***X***

KeyGen() → (*SK, PK*)
– Return OWP.KeyGen()

Sign(*SK, M*) → **σ**
– Return OWP.Invert(*SK*, Hash(*M*))

"Hash-and-sign paradigm"

Verify(*PK*, *M*, **σ**) → {0,1}
– Return "1" iff Hash(M) == OWP.F(*PK*, **σ**) // OWP.F(*PK*, OWP.Invert(*SK*, Hash(*M*))
// = Hash(*M*) if signer is honest

Signatures from trapdoor one-way permutation

Correctness: Want to show that for all (*SK, PK*) ← KeyGen(), messages *M*:

- Verify(*PK*, *M*, Sign(*SK*, *M*) =1
  Iff OWF.F(*PK*, OWP.Invert(*SK*, Hash(*M*))) == Hash(*M*) // By construction
  Iff Hash(*M*) == Hash(*M*) // By Trapdoor OWP correctness

Security: Show if attacker forges signature ⇒ Attacker inverts trapdoor OWP

- Will not go through the security argument here
- Requires modeling the hash function as “ideal” hash function = random oracle
  - Needs much more than collision resistance / one-wayness
  - In practice, we use SHA256

Very nice!

But how you do build a trapdoor one-way permutation?

The RSA Trapdoor One-Way Permutation



The RSA trapdoor one-way permutation

- Described in a 1977 paper by Ron Rivest, Adi Shamir, Len Adleman
  - First construction of public-key encryption
  - First construction of digital signatures
  - **Both** (!) built from the same trapdoor one-way permutation
- RSA had a company that commercialized public-key crypto
  - Tremendously influential in crypto standards (PKCS series)
  - Verisign spun out of RSA
- The security of RSA depends on factoring being hard
- The construction requires a little bit of number theory
  - I will show one variant of RSA that I like because it is totally self contained

Construction: RSA trapdoor one-way permutation

KeyGen() → (*SK*, *PK*)

- Pick two big primes p, q such that $p = q = 4 \bmod 5$
- Compute $N \leftarrow pq$
- $SK \leftarrow (p, q)$ $PK \leftarrow N$

– Let $\mathbb{Z}_N^* = \{1, \ldots, N-1\} / \{\text{numbers divisible by } p \text{ or } q\}$
– Input space is $\boldsymbol{X} = \mathbb{Z}_N^*$ // Slightly annoying since it depends on the *PK*

F($PK = N, x \in \mathbb{Z}^*{}_N$) → x'

- Return $x' \leftarrow x^5 \bmod N$

// So simple! The standard version uses $x^3 \bmod N$ instead.

Invert($SK, x' \in \mathbb{Z}^*{}_N$)

- Let $d \leftarrow 1 + (|\mathbb{Z}_N^*| - 4)/5$
- Return $(x')^d \bmod N$

// By construction, d will be an integer

Note: All algorithms are efficient

Construction: RSA trapdoor one-way permutation

KeyGen() → (*SK, PK*)

- Pick two big primes *p,q* such that $p = q = 4 \bmod 5$
- Compute $N \leftarrow pq$
- $SK \leftarrow (p,q)$ $PK \leftarrow N$

– Let $Z^*_N = \{1, \ldots, N - 1\}$ / {numbers divisible by *p* or *q*}
– Input space is $\boldsymbol{X} = Z^*_N$ // Slightly annoying since it depends on the *PK*

F($PK = N, x \in Z^*_N$) → *x'*

- Return $x' \leftarrow x^5 \bmod N$ // So simple! A common variant uses $x^3 \bmod N$ instead.

Invert($SK, x' \in \boldsymbol{X}$)

- Let $d \leftarrow 1+(|Z^*_N|-4)/5$ // *By construction, d will be an integer*
- Return $(x')^d \bmod N$

Note: All algorithms are efficient

Two facts we will need

Let $\mathbb{Z}_N^* = \{1, \ldots, N-1\} / \{\text{numbers divisible by } p \text{ or } q\}$

Claim 1: $|\mathbb{Z}_N^*| = (p-1)(q-1)$.

Proof: Numbers divisible by p or q are:

$p, 2p, 3p, \ldots, (q-1)p = q-1$ of them

$q, 2q, 3q, \ldots, (p-1)q = p-1$ of them

$$|\mathbb{Z}_N^*| = pq - (q-1) - (p-1) - 1$$
$$= (p-1)(q-1)$$

Claim 2: For all $x \in \mathbb{Z}_N^*$, we have $x^{|\mathbb{Z}_N^*|} = 1 \bmod N$. ("Euler's Theorem")

"Proof:" Take product of $\{1, 2, 3, \ldots N-1\} = B \bmod N$

Take product of $\{x, 2x, 3x, \ldots, x(N-1)\} = B \cdot x^{|\mathbb{Z}_N^*|} \bmod N$

Then $B = B \cdot x^{|\mathbb{Z}_N^*|} \bmod N$

$\Rightarrow \quad 1 = x^{|\mathbb{Z}_N^*|} \bmod N.$

Two facts we will need

Let Z^*_N = {1, …, N - 1} / {numbers divisible by p or q}

Claim 1: $|Z^*_N| = (p - 1)(q - 1)$

Proof: – Numbers divisible by p or q are: $p, 2p, 3p, \ldots, (q-1)p$ = q-1 of them
$q, 2q, 3q, \ldots, (p-1)q$ = p-1 of them

– $|Z^*_N| = pq - (q - 1) - (p - 1) - 1$
$= (p - 1)(q - 1)$

Claim 2: $\text{For all } x \in \mathbb{Z}_N^*: \quad x^{|\mathbb{Z}_N^*|} = 1 \bmod N.$

"Proof:" – Take product of {1, 2, 3, … N - 1} = B mod N
– Take product of {x, 2x, 3x, …, x(N-1)} = $x^{|\mathbb{Z}_N^*|}$ B mod N
– Then B = B $x^{|\mathbb{Z}_N^*|}$ mod N
⇒ 1 = $x^{|\mathbb{Z}_N^*|}$ mod N

RSA Properties

Correctness: Need to show that for all $x \in \mathbb{Z}_N^*$, Invert(*SK*, F(*PK*, *x*)) = *x*.

$$(x^5)^d = x^{5(1+\frac{|\mathbb{Z}_N^*|-4}{5})} = x^{5+|\mathbb{Z}_N^*|-4} = x \cdot x^{|\mathbb{Z}_N^*|} = x \mod N$$

Security:

- RSA Problem: Given N, $x^5 \bmod N$, for random $x \leftarrow_R \mathbb{Z}_N^*$, find x.
  - $e = 5$ is the "public exponent" here. Common choices are $e = 3, e = 65537$
- We assume that the RSA problem is hard
  - Given factors of N, we can compute d ⇒ Solve RSA problem
- Has withstood 45+ years of cryptanalysis
  - "Breaks" of RSA are in misuse of RSA trapdoor one-way permutation

RSA Properties

Correctness: Need to show that for all x in Z^*_N, Invert(SK, F(PK, x)) = x.

$$(x^5)^d = x^{5(1+\frac{|\mathbb{Z}_N^*|-4}{5})} = x^{5+|\mathbb{Z}_N^*|-4} = x \cdot x^{|\mathbb{Z}_N^*|} = x \mod N$$

Security:

- RSA Problem: Given N, x^5 mod N, for random $x \leftarrow Z^*_N$, find x.
  - e=5 is the "public exponent" here. Common choices are e=3, e=65537
- We assume that the RSA problem is hard
  - Given factors of N, we can compute d ⇒ Solve RSA problem
- Has withstood 45+ years of cryptanalysis
  - "Breaks" of RSA are in misuse of RSA trapdoor one-way permutation

Recap: Digital signatures from RSA

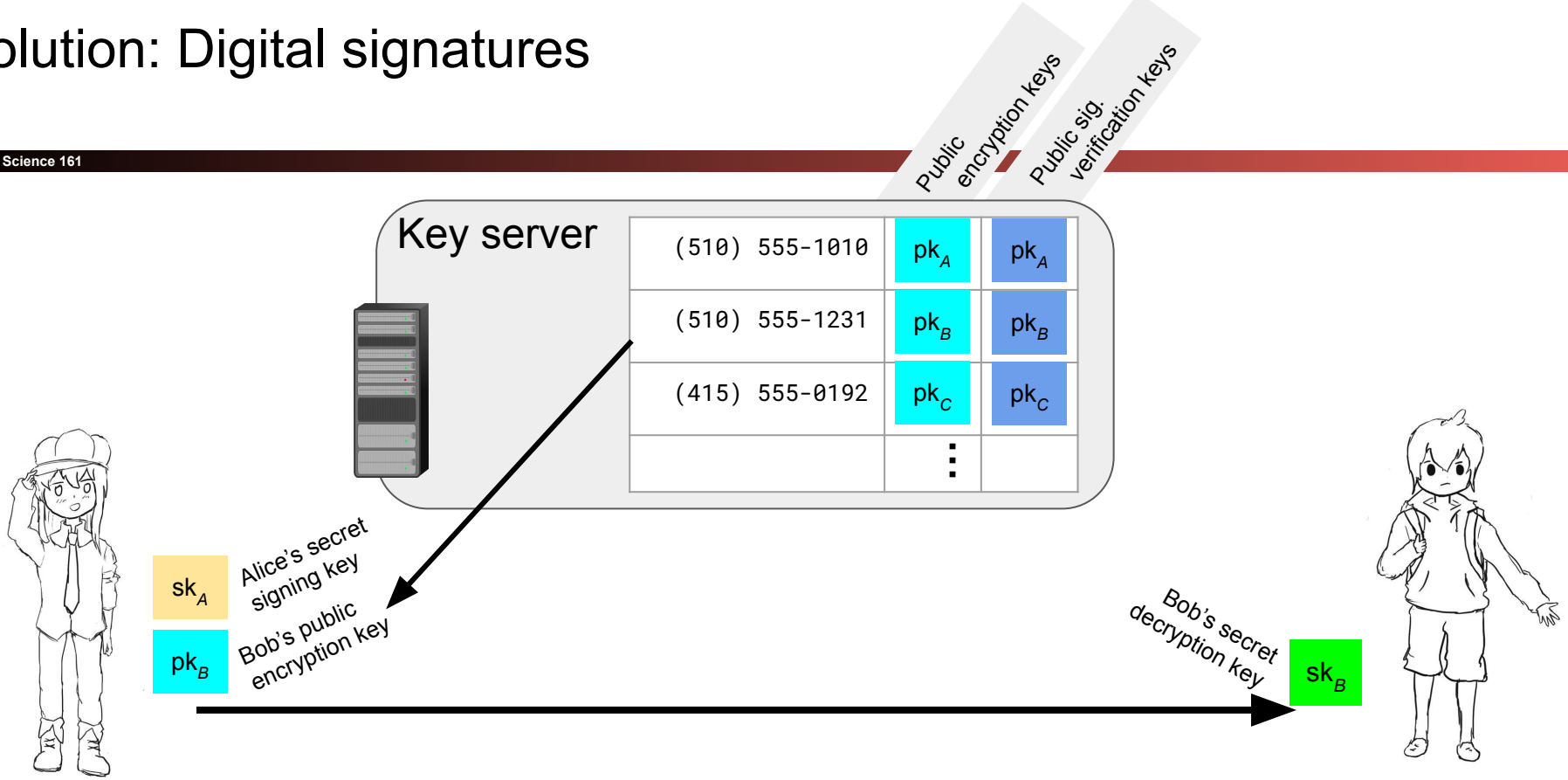
Parameters: – RSA trapdoor OWP (RSA.KeyGen, RSA.F, RSA.Invert)
– Cryptographic hash function Hash : $\{0,1\}^* \rightarrow Z^*_N$

KeyGen() → (*SK, PK*) // Pick big primes *p, q such that…*
– Return RSA.KeyGen() // Compute $N \leftarrow pq$

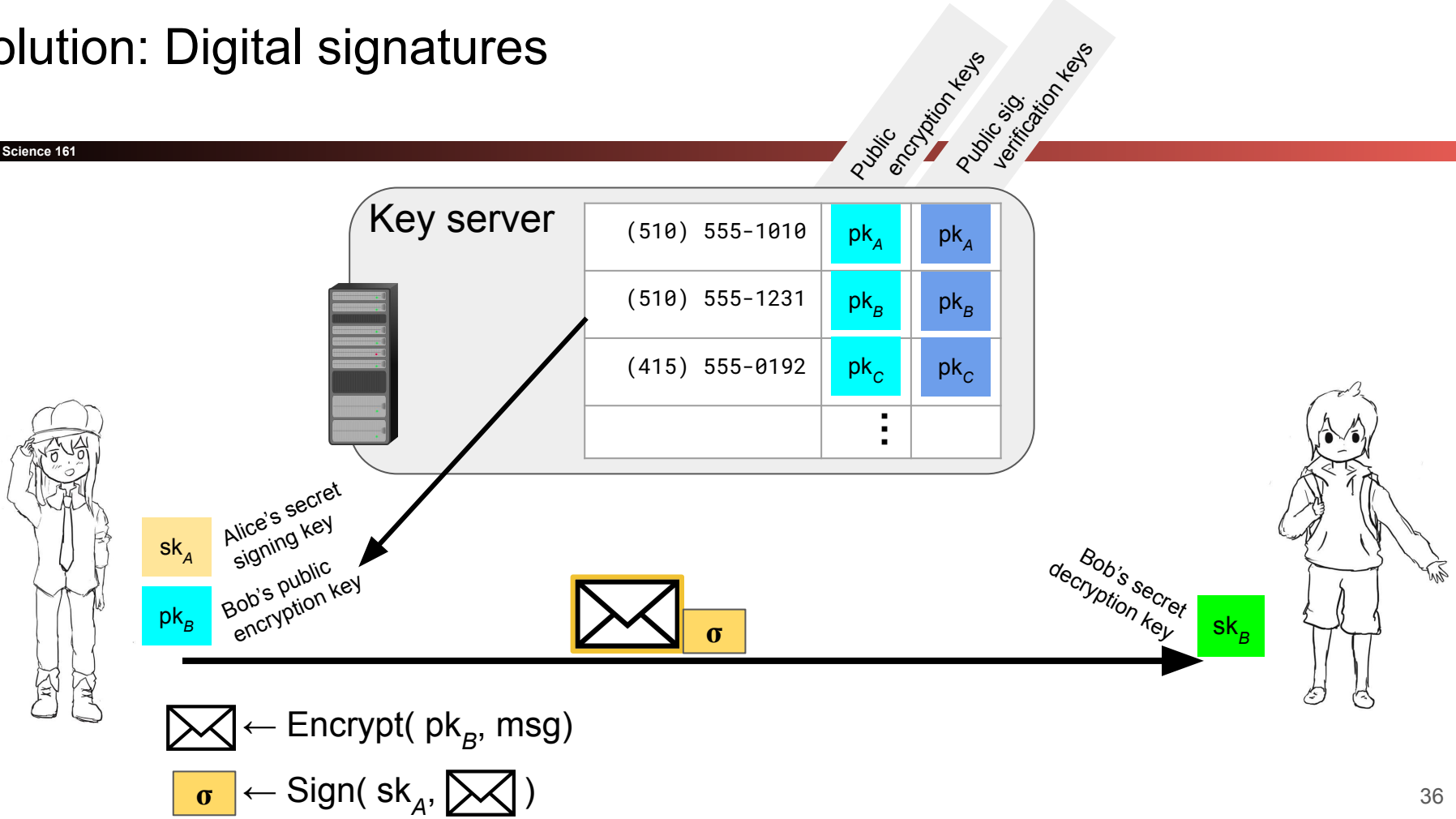
Sign(*SK, M*) → σ
– Return RSA.Invert(*SK*, Hash(*M*)) // Compute Hash(*M*)^d mod *N*

Verify(*PK*, *M*, σ) → {0,1}
– Return "1" iff Hash(M) == RSA.F(*PK*, σ) // Test σ^5 mod *N* == Hash(*M*) ?

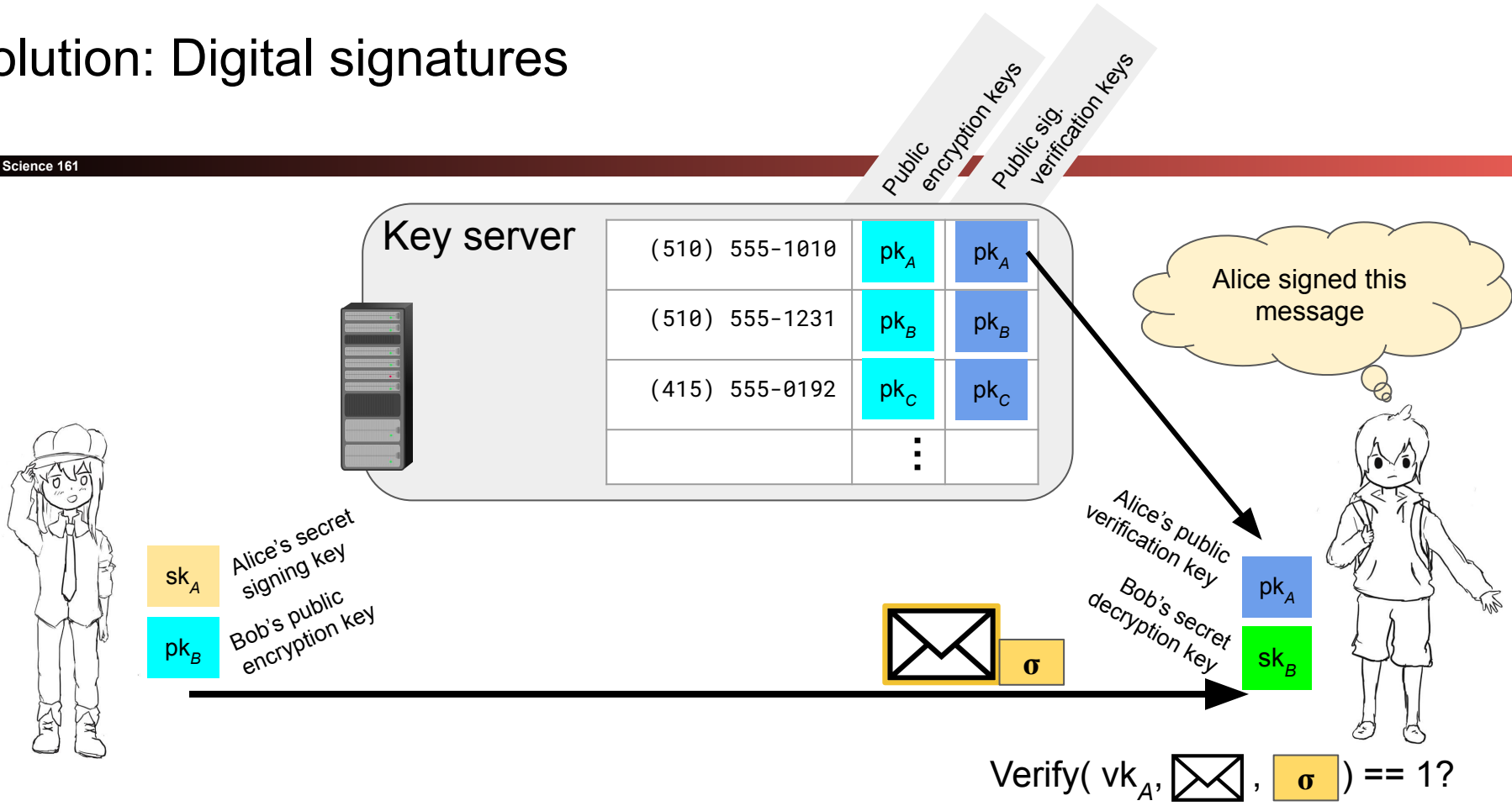
Solution: Digital signatures



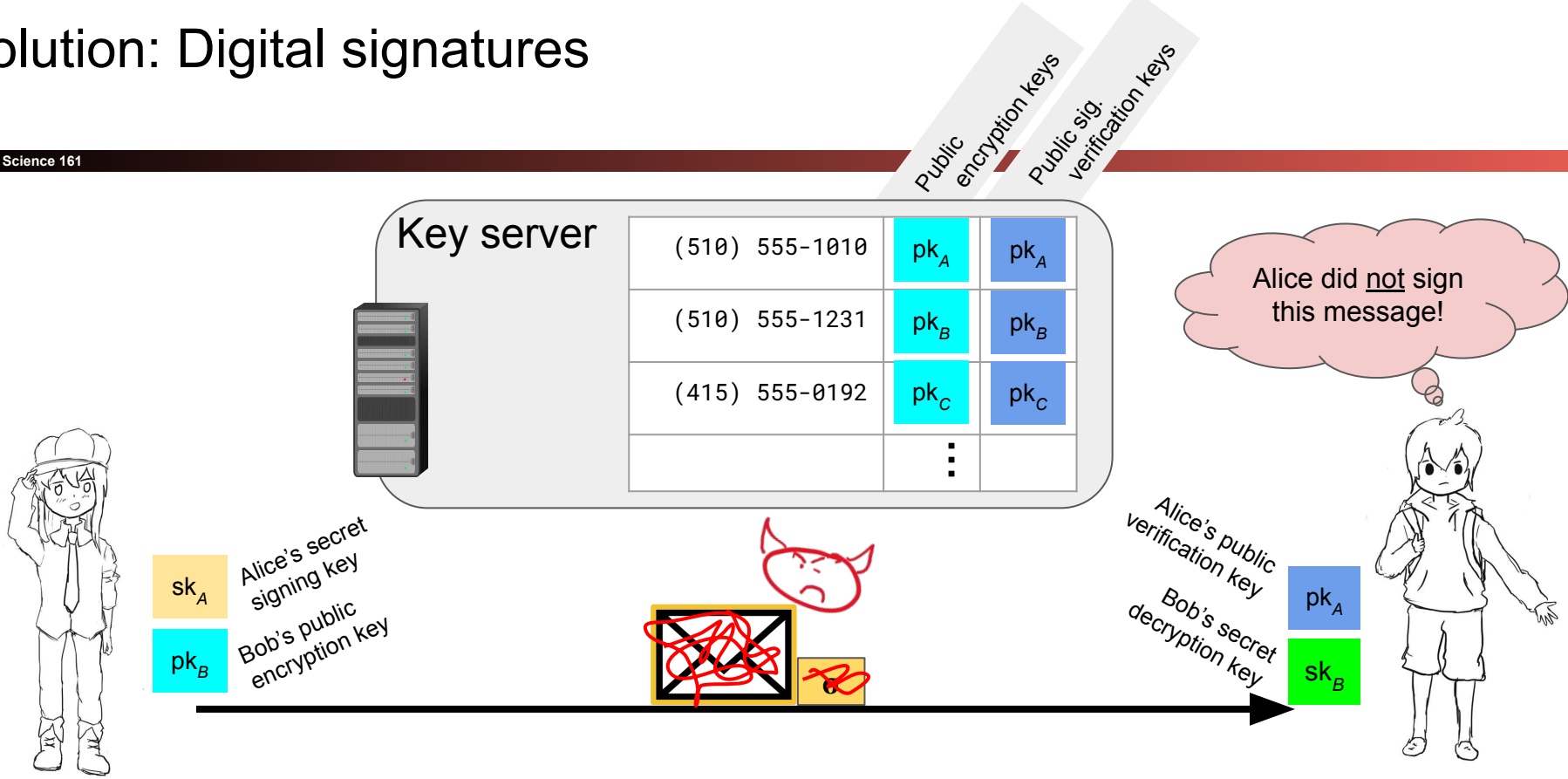
Solution: Digital signatures



Solution: Digital signatures



Solution: Digital signatures



Summary: Digital signatures

- A public-key integrity primitive
  - Alice signs a message with her secret key
  - Anyone can verify that Alice signed // Using Alice's public verification key
- Signatures are used <u>everywhere</u> on the internet
  - SSH, TLS, git commits, Linux packages, …
- There are many constructions of digital signatures
  - RSA signatures are built from the RSA trapdoor one-way permutation

Next time: Crypto implementation bugs.